Simple Java Programs — Class, Methods, Packages, and the Edit-Compile-Run Cycle

Object-Oriented Programming — Week 2 (Java Fundamentals, Chapter 2)

Formatted by: Heisenberg 😎

> Lecture by Dr. Deva Siddharth. This session builds on the previous introduction to OOP (object, class, inheritance, polymorphism, abstraction, encapsulation) and dives into how an actual Java program is structured, written, compiled, and run.

Index

- A First Look at a Java Program — meet `Ch2Sample1.java`, the "hello world" that pops open a window instead of just printing text.
- Data Types, Variables, Operators, and Expressions — Java's box sizes for numbers, from the tiny `byte` to the roomy `double`.
- Standard Output — how to nag the console into showing you things with `System.out`.
- Taking Input with Scanner — teaching your program to actually listen for once.
- Packages and Subpackages — Java's folder-within-a-folder system, minus the desktop clutter.
- Class Declaration — the one rule every Java program must obey: thou shalt have a class.
- Method Declaration — dissecting `public static void main` word by word until it stops looking scary.
- A Program Template for a Simple Java Program — the skeleton you can reuse for literally any Java file.
- The Edit-Compile-Run Cycle — from human-readable code to bytecode to an actual running window.
- Execution Errors — what to blame when the window refuses to show up.
- References

A First Look at a Java Program

Every Java program begins life as a **class**, and the very first example used in this course is a program that opens a small window on screen. The file is named `Ch2Sample1.java`.

A critical rule of Java: **the file name must match the name of the main class**, and both must start with a capital letter. If the class is `Ch2Sample1`, the file must be `Ch2Sample1.java` — not `ch2sample1.java`, not `Sample1.java`. This pairing is not a style preference; the compiler enforces it.

```
/*
   Chapter 2 Sample Program: Displaying a Window
   File: Ch2Sample1.java
*/

class Ch2Sample1 {

    public static void main(String[] args) {

        javax.swing.JFrame myWindow;

        myWindow = new javax.swing.JFrame();

        myWindow.setSize(300, 200);
        myWindow.setTitle("My First Java Program");
        myWindow.setVisible(true);
    }

}
```

Reading this line by line:

- **No import statement** is used here. Instead, the class is referred to by its **fully qualified name**: `javax.swing.JFrame`. This works, but it's verbose — every time the class is used, the entire package path has to be written out.
- `myWindow` is an **object** created from the `JFrame` class. `JFrame` is a *predefined* (standard library) class — nobody in this course wrote it — while `Ch2Sample1` is a *user-defined* class.
- `setSize(300, 200)` fixes the window's dimensions, `setTitle(...)` labels it, and `setVisible(true)` actually makes it appear. Without that last call, the window object would exist in memory but never be shown.
- `javax.swing` is the package that supplies GUI ("window-drawing") functionality — this is why it's needed specifically for a program that displays a window, unlike a plain console program.

> **Pro-Tip:** A common exam trap is forgetting that `setVisible(true)` is *required* — creating a `JFrame` object alone does not display anything. The object exists, but it stays invisible until told otherwise.

The overall structure to internalize from this first example: **define a class → define the main method → write the instructions that fulfill the program's purpose** (here, popping open a window with a message).

Data Types, Variables, Operators, and Expressions

Like any programming language, Java has a fixed set of primitive data types, each with a specific storage size (measured in bits):

Type	Size	Example
`byte`	8-bit	`100`
`short`	16-bit	`2000`
`int`	32-bit	`5000`
`long`	64-bit	`100000L`
`float`	32-bit	`5.4f`
`double`	64-bit	`7.56`
`char`	16-bit	`'A'`
`boolean`	1 bit (logical)	`true`

Along with these types, a program needs **variables** (named storage, like `a`, `b`, `c`), **operators** (`+`, `*`, `>`, and so on) to act on them, and **expressions** that combine variables and operators into a computable statement.

Worked Example

```
int a = 10;
int b = 5;

System.out.println(a+b);
System.out.println(a*b);
System.out.println(a>b);
```

```
Output

15
50
true
```

Walking through this: `a` and `b` are `int` variables holding `10` and `5`. Three separate `System.out.println` statements are used — the Java equivalent of C's `printf` — each ending, as every Java statement must, with a semicolon.

- **First statement** (`a+b`): addition → `10 + 5 = 15`.
- **Second statement** (`a*b`): multiplication → `10 * 5 = 50`.
- **Third statement** (`a>b`): a condition check → is `10` greater than `5`? Yes, so the boolean result is `true`. (If the values were swapped — `a = 5`, `b = 10` — this same statement would print `false`.)

`println` specifically means *print, then move to the next line* — which is why the three outputs appear stacked vertically rather than run together on one line.

This can be done with or without Swing: the earlier window-drawing example used the `javax.swing` package for a GUI, while this example is **plain "core" Java** — no window, just console output. Both are valid; the choice depends on whether the program needs a graphical interface or just a text console.

Standard Output

When a program computes a result, it needs a way to show that result to the user. The most common mechanism in Java is the **console window**, also called the **standard output window**.

Output is sent to the console via `System.out`. Here, `System` is a class, and it contains a public data member named `out`, which is an instance of the `PrintStream` class — a `PrintStream` object permanently tied to the console window (there is exactly one console window per running program). Any data handed to `System.out` appears there. This overall technique — printing via `System.out` — is called **standard output**.

```
System.out.print("Hello, Dr. Caffeine.");
```

Executing this produces:

```
Hello, Dr. Caffeine.
```

(Depending on the development environment, additional system text like `Press any key to continue...` may also appear — that's just the terminal, not part of the program's own output, and can be ignored.)

`print` vs. `println`

`print` leaves the cursor on the same line; `println` moves to a new line afterward. For a program that must display several separate lines of text, `println` is the natural choice:

```
System.out.println("How do you do? ");
System.out.println("My name is ");
System.out.println("Seattle Slew.");
```

```
Output

How do you do?
My name is
Seattle Slew.
```

Each of the three lines is printed by its own `println` call, and because `println` automatically advances to a new line, the three pieces of text stack neatly rather than running together.

Taking Input with Scanner

The examples so far have used **hard-coded** input — values like `a = 10` and `b = 5` were baked directly into the source code. Real programs, however, usually need to accept input that changes every time the program runs. For this, Java provides the **`Scanner`** class.

`Scanner` belongs to the `java.util` package, so it must be imported before it can be used — similar in spirit to including a header file in C:

```
import java.util.Scanner;

public class Demo {

  public static void main(String args[]) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter age:");

    int age = sc.nextInt();

    System.out.println(age);

    sc.close();
  }
}
```

Breaking this down:

1. `import java.util.Scanner;` makes the `Scanner` class available. Without this import, the class name would need to be written out in full (`java.util.Scanner`) every time it's used.
2. `Scanner sc = new Scanner(System.in);` creates a `Scanner` object named `sc`, wired to read from `System.in` (standard input — normally the keyboard).
3. `sc.nextInt()` waits for the user to type an integer and stores it in the variable `age`.
4. `sc.close()` releases the scanner once input is no longer needed — good practice, though a program will often still run without it.

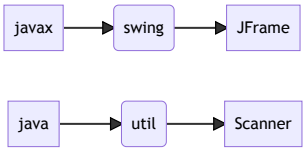
If the user enters `50` when prompted, the program echoes `50` straight back via `System.out.println(age)`. The key idea: whatever value the user types in becomes the value stored in `age`, and the program's output reflects exactly that.

> **Pro-Tip:** `Scanner` is a small class with a big package chain behind it — `java` (main package) → `util` (subpackage) → `Scanner` (class). Understanding this hierarchy is exactly the setup needed for the next topic: packages and subpackages.

Packages and Subpackages

A **package** is Java's way of organizing related classes — conceptually similar to a directory holding files. A package can itself contain **subpackages**, forming a hierarchy of packages nested inside one another.

To refer to a class buried in a deeply nested package, dots chain the names together: `java.util.Scanner` reads as *the `Scanner` class, found in the `util` subpackage, found in the `java` main package*. Likewise, `javax.swing.JFrame` means *the `JFrame` class, in the `swing` subpackage, in the `javax` main package*.



Naming convention: package names conventionally start with a lowercase letter (`javax` , `util`), while class names start with an uppercase letter (`JFrame` , `Scanner`). This is exactly why, when reading `javax.swing.JFrame` , it's clear at a glance which segments are packages and which final segment is the class.

The Import Statement

Writing out a fully qualified name (`javax.swing.JFrame`) every time a class is used is tedious. The **import statement** solves this: it allows a program to refer to classes in a designated package *without* the fully qualified name.

```
import javax.swing.JFrame;
```

After this line, the program can simply write `JFrame` instead of `javax.swing.JFrame` . Note that, like every Java statement, the import line ends with a semicolon.

If a program needs several classes from the same package, importing each individually becomes repetitive:

```
import javax.swing.JFrame;
import javax.swing.JButton;
import javax.swing.JLabel;
```

Instead, the asterisk (`*`) notation imports *all* classes in that package at once:

```
import javax.swing.*;
```

This is exactly the technique used earlier: `import javax.swing.*;` at the top of `Ch2Sample1.java` makes `JFrame` (and every other Swing class) directly accessible, so the code can write:

```
class Ch2Sample1 {

    public static void main(String[] args) {

        JFrame myWindow;

        myWindow = new JFrame();

        myWindow.setSize(300, 200);
        myWindow.setTitle("My First Java Program");
        myWindow.setVisible(true);
    }

}
```

— identical behavior to the earlier fully-qualified version, just shorter and easier to read.

Class Declaration

A Java program is composed of **one or more classes** — it is not mandatory to have only a single class (recall from the inheritance discussion in the previous session, where superclasses and subclasses coexist in the same program). Some of these classes are **predefined** (part of the Java standard library, like `JFrame`), while others are **user-defined** (written by the programmer, like `Ch2Sample1`).

To define a new class, it must be *declared*. The general syntax is:

```
class <class name> {
    <class member declarations>
}
```

- `<class name>` is the name of the class.
- `<class member declarations>` is a sequence of members — each member is either a **data value** (a variable) or a **method**.
- `class` is a reserved word marking the start of a class declaration.
- Any valid identifier that is *not* a reserved word can be used as the class name.

Matched against the running example:

```
class Ch2Sample1 {

    public static void main(String[] args) {

        JFrame myWindow;

        myWindow = new JFrame();

        myWindow.setSize(300, 200);
        myWindow.setTitle("My First Java Program");
        myWindow.setVisible(true);
    }

}
```

Every program must include at least one class. A program can absolutely have more than one, but never zero.

Method Declaration

Inside a class live its **methods**. The general syntax for declaring one is:

```
<modifiers> <return type> <method name> ( <parameters> ) {
    <method body>
}
```

- **`<modifiers>`** — a sequence of terms describing different kinds of methods (such as `public`, `static`).
- **`<return type>`** — the type of data value the method sends back when it finishes (could be `int`, `float`, `double`, `void` for nothing, and so on).
- **`<method name>`** — the identifier used to call the method.
- **`<parameters>`** — the sequence of values passed into the method; a method may take zero, one, or many parameters.
- **`<method body>`** — the sequence of instructions the method actually performs (addition, multiplication, printing, or any other operation).

Matched against the `main` method used throughout these examples:

Component	Value in the example
Modifier	`public`
Modifier	`static`
Return type	`void`
Method name	`main`
Parameter	`String[] args`
Method body	everything inside the `{ }`

```
public static void main( String[] args ) {
    JFrame myWindow;
    myWindow = new JFrame();
    myWindow.setSize(300, 200);
    myWindow.setTitle("My First Java Program");
    myWindow.setVisible(true);
}
```

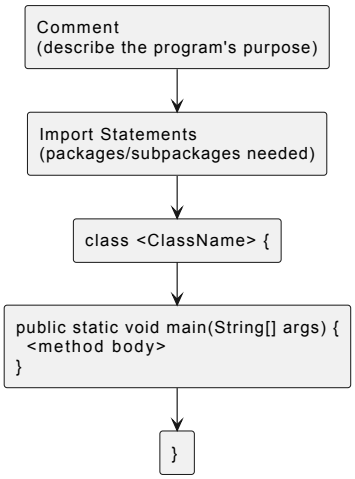
Here, `public` and `static` are both modifiers, `void` is the return type (meaning the method returns nothing), `main` is the method name, and `String[] args` is the parameter — an *array* of strings, which is why the square brackets appear.

Note that `main` is special only in the sense that Java always looks for it as the entry point of the program — structurally, it is declared exactly like any other method. A user-defined method for, say, addition, would follow the same six-part shape: modifier, return type, method name, parameters, and method body — just with `int add(int a, int b) { return a + b; }` in place of `main`.

> **Pro-Tip:** Remember the method declaration in six parts: **modifier → (modifier) → return type → method name → parameters → method body.** This shape never changes, whether the method is `main` or something entirely custom.

A Program Template for a Simple Java Program

Putting everything together, every Java program — however big or small — follows the same overall template:



1. **Comment** — describes what the program does and why, so any reader can understand its purpose without re-deriving it from the code.
2. **Import statements** — a sequence of imports for whatever packages and subpackages the program needs. If no external package is required (as in the plain arithmetic example earlier), this section is simply omitted — it is optional, not mandatory.
3. **Class name** — a descriptive name for the main class (the class containing `main`), matching the file name.
4. **Method body** — the sequence of instructions that do the actual work, inside `main` (or other methods).

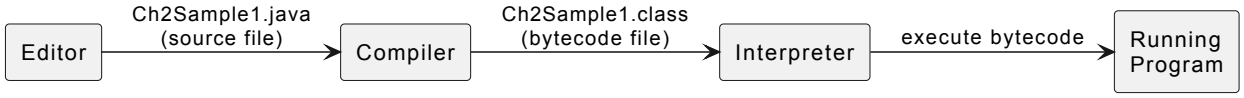
Applying the template to `Ch2Sample1.java` makes the mapping explicit:

- *Comment:* the `/* Chapter 2 Sample Program: Displaying a Window ... */` block.
- *Import:* deliberately **absent** in the fully-qualified version of this example — it's optional, and here fully qualified names (`javax.swing.JFrame`) are used instead.
- *Class name:* `Ch2Sample1` .
- *Method + body:* `public static void main(String[] args) { ... }` containing the `JFrame` creation and display logic.

The one universal truth across every Java program is this: **there must be a class, and there must be a main method** — everything else (imports, extra methods, comments) is added as the program's needs require.

The Edit-Compile-Run Cycle

Turning source code into a running program always follows three steps.



Step 1 — Edit. Type the program using an editor (a `vi` editor, a `gedit` editor, or any text/code editor) and save it to a file. The filename must use the name of the *main class*, suffixed with `.java` . This human-readable file is called the **source file** — here, `Ch2Sample1.java` .

Step 2 — Compile. Compile the source file. Many compilers require creating a project file and placing the source file inside it before compiling; alternatively, the source file can sometimes be compiled directly. When compilation succeeds, the compiler produces a **compiled version** of the source file called **bytecode**, stored in a **bytecode file**. The bytecode file's name shares the same prefix as the source file, but with the suffix `.class` instead of `.java` — so `Ch2Sample1.java` compiles to `Ch2Sample1.class` .

Step 3 — Run. Execute the bytecode file. A **Java interpreter** goes through the bytecode and executes its instructions one by one. If the program is error-free, the intended result appears — in this example, a window of the requested size (say, `300 × 200`) pops up on screen. If the program contains errors, it does not execute.

Put together: **Editor → (source file, `.java`) → Compiler → (bytecode file, `.class`) → Interpreter → Running Program.** Every Java program, regardless of size or complexity, follows exactly this pipeline.

Execution Errors

If an error occurs while the program is running, the **interpreter** catches it and halts execution. An error detected at this stage — during the interpreter's pass through the bytecode — is called an **execution error**.

If the expected result doesn't appear:

1. Go back through the previous steps and verify the program was entered correctly (check the logic and syntax).
2. If the expected result *still* doesn't appear after that, the development environment itself is most likely not set up correctly.

In short, a failure to see the expected output traces back to one of exactly two causes: **incorrect program logic**, or **an incorrectly configured development environment** — and it's worth checking both rather than assuming it must be one or the other.

> **Pro-Tip:** This distinction — "is it my code, or is it my setup?" — is a genuinely useful debugging habit well beyond this course. When a program that "should" work doesn't, isolating whether the problem is the logic or the environment saves a lot of wasted troubleshooting time.

References

1. *A Comprehensive Introduction to Object-Oriented Programming With Java* by C. Thomas Wu, ISBN-13: 978-0071285858
2. *Java: The Complete Reference*, 13th Edition, by Herbert Schildt, ISBN-13: 978-9355326478